

**Manara University**

**Department of Robotics and Intelligent Systems Engineering**

**Machine Learning — Practical Project Documentation**

---

# **Bayesian Medical Risk Analyzer**

A Gaussian Naïve Bayes Desktop Application  
for Binary Medical Risk Classification

---

## **Project Team**

Beilassan Hdewa - Lana Alwazzeh - Zain Alabidin Shbani

Supervised by:  
Dr. Eng. Mohammad Mashi

# Table of Contents

- 1 Project Overview**
- 2 Theoretical Background**
  - 2.1 Gaussian Naïve Bayes
  - 2.2 Log-Space Inference & Numerical Stability
  - 2.3 Evaluation Metrics
  - 2.4 Variance Smoothing
- 3 System Architecture**
  - 3.1 Class Hierarchy
  - 3.2 Data Flow
- 4 Component Reference**
  - 4.1 AppConstants
  - 4.2 TrainingSample & ModelSettings
  - 4.3 GaussianClassModel
  - 4.4 TrainedBayesianClassifier
  - 4.5 ModelTrainer
  - 4.6 ModelEvaluator
  - 4.7 DiagnosticSession
  - 4.8 RiskBar (Custom Control)
  - 4.9 Form1 (Main UI)
- 5 User Interface Walkthrough**
  - 5.1 Training Tab
  - 5.2 Inference Tab
  - 5.3 Analytics Tab
  - 5.4 Model Lab Tab
- 6 Hyperparameter Guide**
- 7 Dataset Specification**
- 8 Model Persistence (.mdl Format)**
- 9 Validation Results**
- 10 Limitations & Future Work**
- 11 References**

# 1. Project Overview

The Bayesian Medical Risk Analyzer is a single-file WinForms desktop application developed in C# targeting .NET Framework 4.0. It implements a complete Gaussian Naïve Bayes (GNB) binary classification pipeline for the task of distinguishing healthy individuals from at-risk patients based on four clinical features: fasting plasma glucose, age, body-mass index (BMI), and systolic blood pressure.

The application targets a medical screening scenario where a clinician inputs patient biometric measurements and receives a real-time probabilistic risk assessment. The system is self-contained: training data can be entered manually, loaded from a demo dataset, or imported from a CSV file. The trained model may be serialized to a plain-text .mdl file for later reuse.

## 1.1 Objectives

- Implement a mathematically rigorous Gaussian Naïve Bayes classifier from first principles (no third-party ML library).
- Provide both stratified 80/20 Train/Test Split and Leave-One-Out (LOO) cross-validation for thorough model evaluation.
- Expose all critical hyperparameters (prior, smoothing, decision threshold, class weight) through interactive sliders.
- Deliver real-time probabilistic inference via a responsive WinForms GUI with live-updating charts.
- Persist trained model parameters in a portable plain-text format compatible with the same application version.

## 1.2 Key Features at a Glance

| Feature                | Description                                                                            |
|------------------------|----------------------------------------------------------------------------------------|
| Gaussian Naïve Bayes   | Per-class Gaussian likelihood; log-space arithmetic to prevent underflow               |
| Train/Test Split       | Stratified 80/20 split preserving class ratios in both subsets                         |
| LOO Cross-Validation   | Leave-One-Out evaluation; folds with insufficient class coverage are skipped           |
| Hyperparameter Sliders | Live-tuning of prior probability, variance smoothing, decision threshold, class weight |
| Live Inference         | Posterior probabilities and classification label update on every slider move           |
| Outlier Detection      | Z-score-based detection; warns when the input lies outside the learned distribution    |
| CSV Import / Export    | Load datasets with columns Glucose, Age, BMI, BloodPressure, Status                    |

|                       |                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------|
| Model Save / Load     | Serializes all Gaussian parameters and settings to a versioned .mdl text file             |
| Analytics Dashboard   | Class statistics, confusion matrix bar chart, class-distribution doughnut chart, F1 score |
| P(Patient) vs Glucose | Line chart sweeping glucose 50–250 mg/dL at fixed values of other features                |

## 2. Theoretical Background

### 2.1 Gaussian Naïve Bayes

Naïve Bayes is a probabilistic classifier that applies Bayes' theorem with the 'naïve' conditional independence assumption: each feature is assumed to be statistically independent of every other feature given the class label. For a feature vector  $\mathbf{x} = (x_1, x_2, \dots, x_n)$  and class label  $c$ , the posterior is:

$$P(c | \mathbf{x}) \propto P(c) \cdot \prod_i P(x_i | c)$$

In the Gaussian variant, each per-class conditional likelihood  $P(x_i | c)$  is modelled as a univariate Gaussian distribution whose parameters — mean  $\mu_{i,c}$  and variance  $\sigma_{i,c}^2$  — are estimated from the training data using maximum likelihood estimation (MLE):

$$P(x_i | c) = (1 / \sqrt{2\pi \sigma_{i,c}^2}) \cdot \exp(-(x_i - \mu_{i,c})^2 / (2\sigma_{i,c}^2))$$

The variance is estimated using Bessel's correction (denominator  $n - 1$ ) to produce an unbiased sample variance. A smoothing constant  $\varepsilon$  is then added to every estimated variance to prevent numerical collapse on very small or perfectly separated datasets:

$$\sigma_{i,c}^2 \leftarrow \sigma_{i,c}^2 + \varepsilon, \quad \varepsilon = \lambda \cdot \max(\sigma_{i,c}^2)$$

### 2.2 Log-Space Inference & Numerical Stability

Multiplying many small probabilities leads to arithmetic underflow for high-dimensional feature spaces. The classifier converts all computations to logarithm space. The joint log-score for class  $c$  is:

$$\log \tilde{P}(c | \mathbf{x}) = \log P(c) + \sum_i \log P(x_i | c)$$

Posterior probabilities are recovered via the log-sum-exp trick, which subtracts the maximum log-score before exponentiation, ensuring numerical stability:

$$\log Z = \max(s^c) + \log \sum^c \exp(s^c - \max(s^c)), \quad P(c | \mathbf{x}) = \exp(s^c - \log Z)$$

### 2.3 Evaluation Metrics

The system reports four standard binary-classification metrics derived from the confusion matrix (TP, TN, FP, FN):

| Metric    | Formula                           | Interpretation                                    |
|-----------|-----------------------------------|---------------------------------------------------|
| Accuracy  | $(TP + TN) / (TP + TN + FP + FN)$ | Overall fraction of correct predictions           |
| Precision | $TP / (TP + FP)$                  | Fraction of positive predictions that are correct |
| Recall    | $TP / (TP + FN)$                  | Fraction of actual positives correctly identified |
| F1 Score  | $2 \cdot (P \cdot R) / (P + R)$   | Harmonic mean of precision and recall             |

## 2.4 Variance Smoothing (Laplace-style Regularization)

When training data are scarce, sample variance may collapse to zero for certain features, making the Gaussian likelihood undefined. The application adds an epsilon term proportional to the maximum observed variance, scaled by the user-supplied smoothing coefficient  $\lambda \in [0, 0.1]$ . This is analogous to Laplace smoothing in discrete Naïve Bayes and prevents degenerate likelihoods.

# 3. System Architecture

## 3.1 Class Hierarchy

The entire application is contained in a single C# source file (BayesianAnalyzer.cs) organized into eleven distinct classes within the BayesianAnalyzer namespace. The separation follows a clean layered architecture:

| Layer          | Class(es)                                                                                                | Responsibility                                                                                                      |
|----------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Entry Point    | Program                                                                                                  | Bootstraps the WinForms application thread                                                                          |
| Constants      | AppConstants                                                                                             | Application-wide magic numbers and string labels                                                                    |
| Data Models    | TrainingSample, ModelSettings, GaussianClassModel, BayesianPrediction, EvaluationReport, TrainTestReport | Immutable value types representing training records, hyperparameters, learned distributions, and evaluation results |
| ML Core        | TrainedBayesianClassifier                                                                                | Stores trained Gaussian parameters; implements Predict() in log-space                                               |
| ML Core        | ModelTrainer                                                                                             | Fits per-class Gaussian distributions from labeled samples; applies variance smoothing                              |
| ML Core        | ModelEvaluator                                                                                           | Executes stratified Train/Test evaluation and LOO cross-validation                                                  |
| Session Façade | DiagnosticSession                                                                                        | Thin orchestrator: Train → Predict →                                                                                |

|                |         |                                                                                     |
|----------------|---------|-------------------------------------------------------------------------------------|
|                |         | Evaluate → Reset lifecycle                                                          |
| Custom Control | RiskBar | Custom Panel-derived progress bar showing posterior probability with color feedback |
| UI             | Form1   | Main WinForms form; builds all tabs, handles events, renders charts                 |

## 3.2 Data Flow

The following describes the complete data flow from user input to classification output:

1. User enters or imports patient records → stored in DataGridView (`_trainingGrid`).
2. `ReadSamplesFromGrid()` parses each row into `TrainingSample` objects (feature vector + label).
3. `ValidateSamples()` checks minimum sample counts before proceeding.
4. `SplitSamples()` performs stratified 80/20 partitioning (Train/Test Split path) or data passes whole (LOO path).
5. `ModelTrainer.Train()` computes per-class mean, Bessel-corrected variance, and prior; applies smoothing epsilon; returns `TrainedBayesianClassifier`.
6. `DiagnosticSession` caches the trained classifier instance.
7. `ModelEvaluator.EvaluateTrainTest()` or `EvaluateLeaveOneOut()` generates an `EvaluationReport` (TP/TN/FP/FN → Accuracy, Precision, Recall, F1).
8. `Form1` refreshes the Analytics tab (stats labels, pie chart, confusion bar chart).
9. On Inference tab, `TrainedBayesianClassifier.Predict()` computes log-joint scores, applies log-sum-exp normalization, compares posterior against decision threshold.
10. `RiskBar` and diagnosis label are updated; `P(Patient)` vs Glucose chart is redrawn.

## 4. Component Reference

### 4.1 AppConstants

A static class grouping all application-wide constants to avoid magic numbers throughout the codebase.

| Constant                        | Value     | Purpose                                              |
|---------------------------------|-----------|------------------------------------------------------|
| <code>PositiveLabel</code>      | "Patient" | String identifier for the at-risk class              |
| <code>NegativeLabel</code>      | "Healthy" | String identifier for the healthy class              |
| <code>MinTotalSamples</code>    | 6         | Minimum dataset size required before training        |
| <code>MinSamplesPerClass</code> | 3         | Minimum per-class sample count for a valid fold      |
| <code>OutlierZScore</code>      | 3.0       | Mean feature z-score threshold for outlier detection |
| <code>MinEpsilon</code>         | 1e-6      | Absolute floor for the smoothing constant epsilon    |

|           |       |                                                |
|-----------|-------|------------------------------------------------|
| MinPrior  | 1e-12 | Floor applied when a class prior would be zero |
| MinWeight | 1e-12 | Floor for the positive-class cost multiplier   |

## 4.2 Training Sample & ModelSettings

TrainingSample is a plain data class holding a double[] Features array and a string Label. ModelSettings encapsulates all hyperparameters that affect training and inference:

| Property            | Type   | Description                                                              |
|---------------------|--------|--------------------------------------------------------------------------|
| UseEmpiricalPrior   | bool   | If true, prior = class frequency; otherwise ManualPositivePrior is used  |
| ManualPositivePrior | double | User-specified prior for the Patient class (0–1)                         |
| VarianceSmoothing   | double | Smoothing coefficient $\lambda$ (slider value / 1000)                    |
| DecisionThreshold   | double | Posterior cutoff above which a sample is labelled Patient (default 0.50) |
| PositiveClassWeight | double | Asymmetric cost multiplier on the Patient log-prior (default 1.00×)      |
| PositiveLabel       | string | Canonical name of the positive class                                     |
| NegativeLabel       | string | Canonical name of the negative class                                     |

## 4.3 GaussianClassModel

Stores all learned parameters for one class after training: sample count, prior probability, and per-feature arrays of mean, variance (with smoothing applied), and standard deviation. Instances are created by ModelTrainer and consumed by TrainedBayesianClassifier.

## 4.4 TrainedBayesianClassifier

The core inference engine. It is constructed by ModelTrainer and becomes immutable after creation. The primary public method is Predict(double[] features), which:

- Iterates over all class models and computes a log-joint score:  $\log P(c) + \sum_i \log N(x_i; \mu_i, c, \sigma_i^2, c)$ .
- Optionally adds  $\log(\text{PositiveClassWeight})$  to the Patient log-score (asymmetric cost).
- Applies log-sum-exp normalization to recover proper posterior probabilities.
- Compares  $P(\text{Patient} | x)$  against DecisionThreshold to assign the predicted label.
- Calls CheckIsOutlier() to flag inputs whose mean z-score exceeds OutlierZScore = 3.0.

## 4.5 ModelTrainer

Implements the `Train(List<TrainingSample>, ModelSettings)` method. Training proceeds in two passes:

- Pass 1 — Groups samples by label; computes per-feature means (arithmetic mean) and sample variances (Bessel-corrected,  $n - 1$  denominator); records the global maximum variance across all classes and features.
- Pass 2 — Computes  $\epsilon = \max(\lambda \cdot \text{globalMaxVariance}, \text{MinEpsilon})$ ; adds epsilon to every variance entry; derives standard deviations. Returns a new `TrainedBayesianClassifier` instance.

If either class is absent from the training set, an `InvalidOperationException` is thrown before any computation begins.

## 4.6 ModelEvaluator (static)

Provides two static evaluation routines:

### **EvaluateLeaveOneOut(samples, settings)**

- Iterates over all  $n$  samples; for each fold  $i$ , trains on the remaining  $n-1$  samples, predicts sample  $i$ .
- Folds where either class would have fewer than `MinSamplesPerClass` (3) examples are silently skipped (`SkippedFolds` counter incremented).
- Returns an `EvaluationReport` with cumulative TP, TN, FP, FN counts and derived metrics.

### **EvaluateTrainTest(train, test, settings)**

- Trains a fresh classifier on the train list; evaluates it on both train and test lists.
- Returns a `TrainTestReport` containing separate `EvaluationReport` objects for the training set and the test set.

## 4.7 DiagnosticSession

A thin façade that owns the single active `TrainedBayesianClassifier` instance and delegates all operations to the ML core classes. Its lifecycle consists of four states: untrained (`IsTrained = false`), trained (after `Train()`), predicting (after `Predict()`), and reset (after `Reset()`, which nulls the classifier reference). It decouples `Form1` from direct dependency on `ModelTrainer` and `ModelEvaluator`.

## 4.8 RiskBar (Custom Control)

A `WinForms Panel` subclass that renders a horizontal filled-bar visualization of a risk percentage. The `SetValue(percent, color)` method adjusts the width of an inner fill panel proportionally to the supplied percentage and changes its background color (green for Healthy, red for Patient). `DoubleBuffered = true` eliminates flicker on resize.



## 4.9 Form1 (Main UI)

The application's single WinForms Form. It contains all UI construction logic (Build\* methods), event handlers, chart refresh routines, CSV import/export, and model serialization. State is maintained through the DiagnosticSession instance (`_session`), the training and test sample lists (`_trainSet`, `_testSet`), and the last evaluation report (`_lastTTReport`). The UI is built entirely in code (no XAML or .resx designer files).

## 5. User Interface Walkthrough

The application window is divided into a header bar, a four-tab content area, and a status bar. The header displays the application title, a feature subtitle, and the current model status (e.g., "Trained (80/20) | Train: 6 Test: 6 | epsilon: 2.53333"). The status bar at the bottom always shows the last evaluation result.

### 5.1 Training Tab

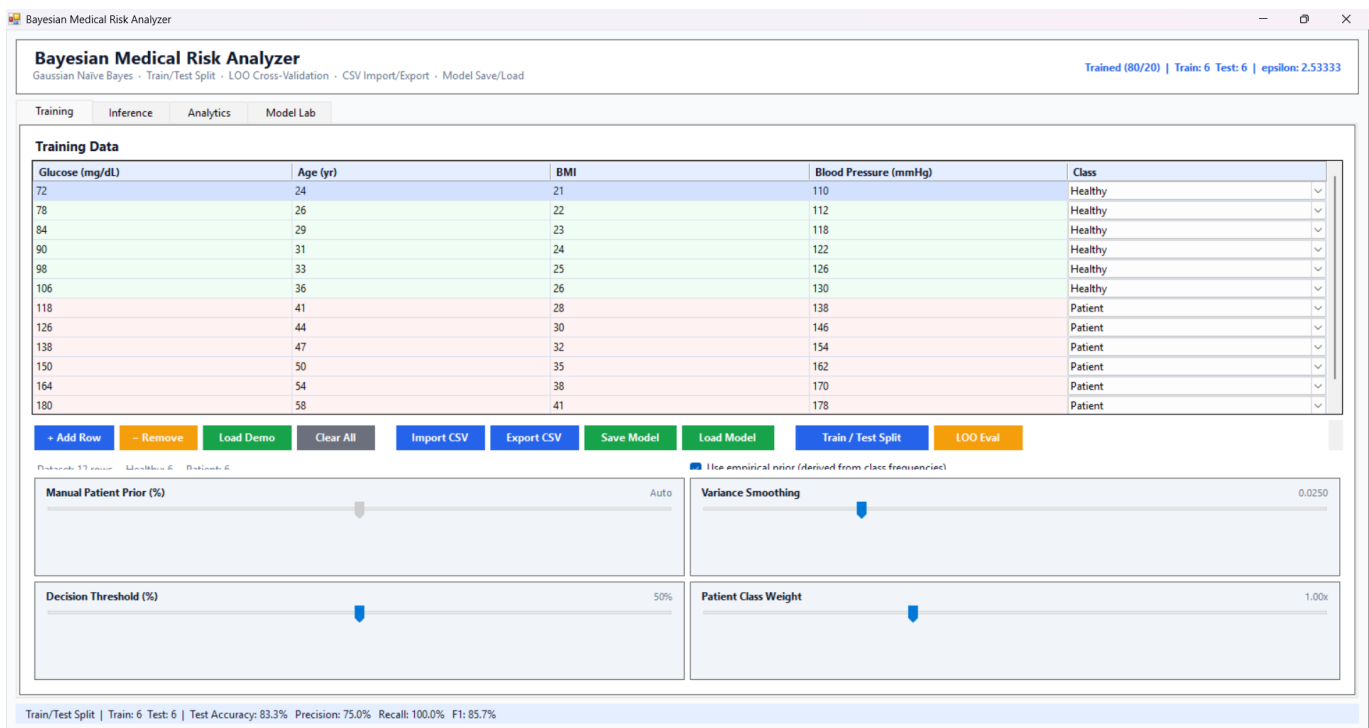


Figure 1. Training Tab showing the data grid with 12 demo samples, training buttons, and hyperparameter sliders

The Training tab is the primary data-entry and model-configuration panel. It is divided vertically into four regions:

- Data Grid — An editable DataGridView with five columns: Glucose (mg/dL), Age (yr), BMI, Blood Pressure (mmHg), and Class (Healthy / Patient dropdown). Rows are color-coded: green tint for Healthy, red tint for Patient.
- Toolbar — Ten action buttons: Add Row, Remove, Load Demo, Clear All, Import CSV, Export CSV, Save Model, Load Model, Train/Test Split, and LOO Eval.
- Summary bar — Shows the current dataset row count and per-class breakdown (e.g., 'Dataset: 12 rows · Healthy: 6 · Patient: 6').
- Hyperparameter sliders — Four controls for Manual Patient Prior, Variance Smoothing, Decision Threshold, and Patient Class Weight (detailed in Section 6).

Clicking Train / Test Split performs a stratified 80/20 split, trains the model on the larger partition, evaluates it on the held-out set, and immediately refreshes all tabs. Clicking LOO Eval trains on all samples then cross-validates using Leave-One-Out.

## 5.2 Inference Tab

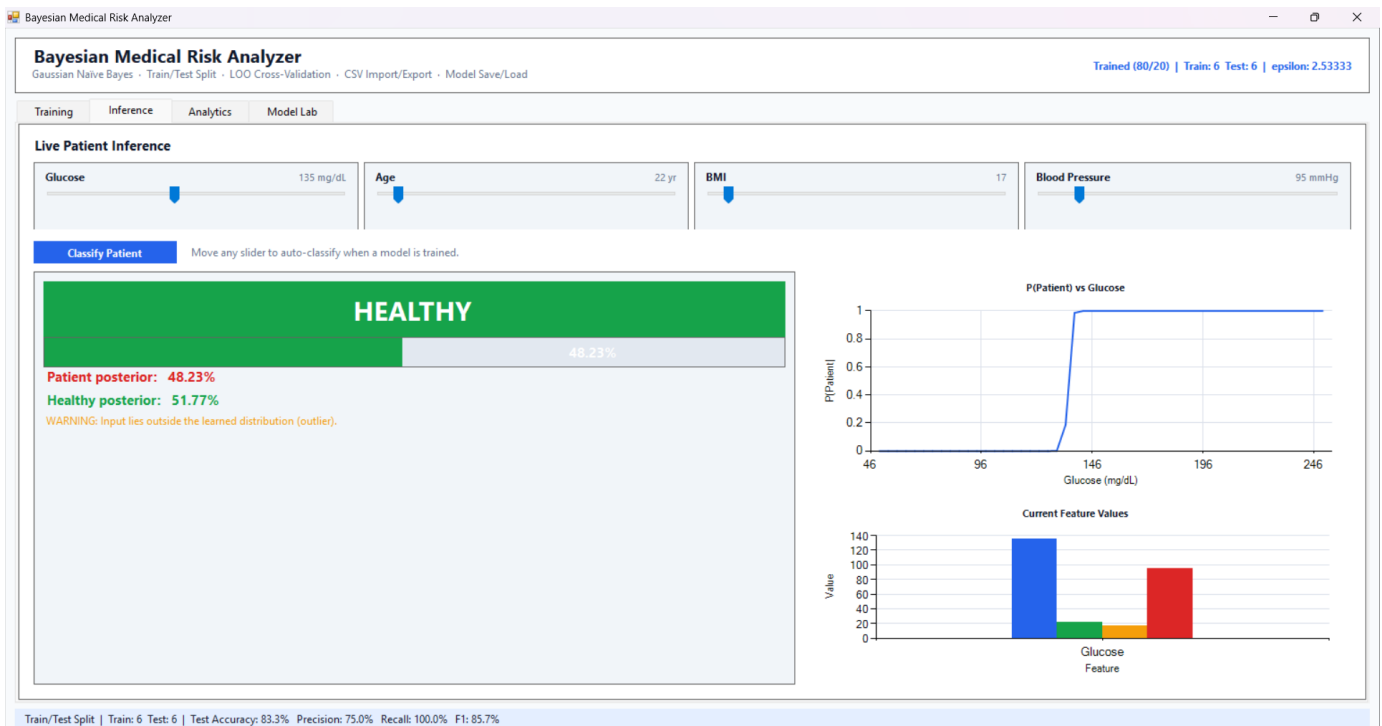


Figure 2. Inference Tab showing a PATIENT classification with 71.91% posterior probability, the  $P(\text{Patient})$  vs Glucose line chart, and the Current Feature Values bar chart.

The Inference tab provides real-time probabilistic risk assessment for a new, hypothetical patient. Four horizontal sliders control the input features:

| Slider         | Range    | Default | Unit              |
|----------------|----------|---------|-------------------|
| Glucose        | 50 – 250 | 120     | mg/dL             |
| Age            | 18 – 90  | 45      | years             |
| BMI            | 15 – 50  | 25      | kg/m <sup>2</sup> |
| Blood Pressure | 80 – 200 | 120     | mmHg              |

Whenever any slider is moved and a model is trained, `RunInference ()` is called automatically. The result panel on the left displays a large diagnosis banner (PATIENT in red, HEALTHY in green), the RiskBar showing the posterior percentage, and separate labels for patient posterior and Healthy posterior. An outlier warning is shown when the input z-score exceeds 3.0.

Two charts update in real time on the right side: (a)  $P(\text{Patient})$  vs Glucose — sweeps glucose from 50 to 250 mg/dL at the current slider values for Age, BMI, and Blood Pressure; (b) Current Feature Values — a column chart showing the raw values of all four features using the application's color scheme (blue, green, orange, red).

### 5.3 Analytics Tab

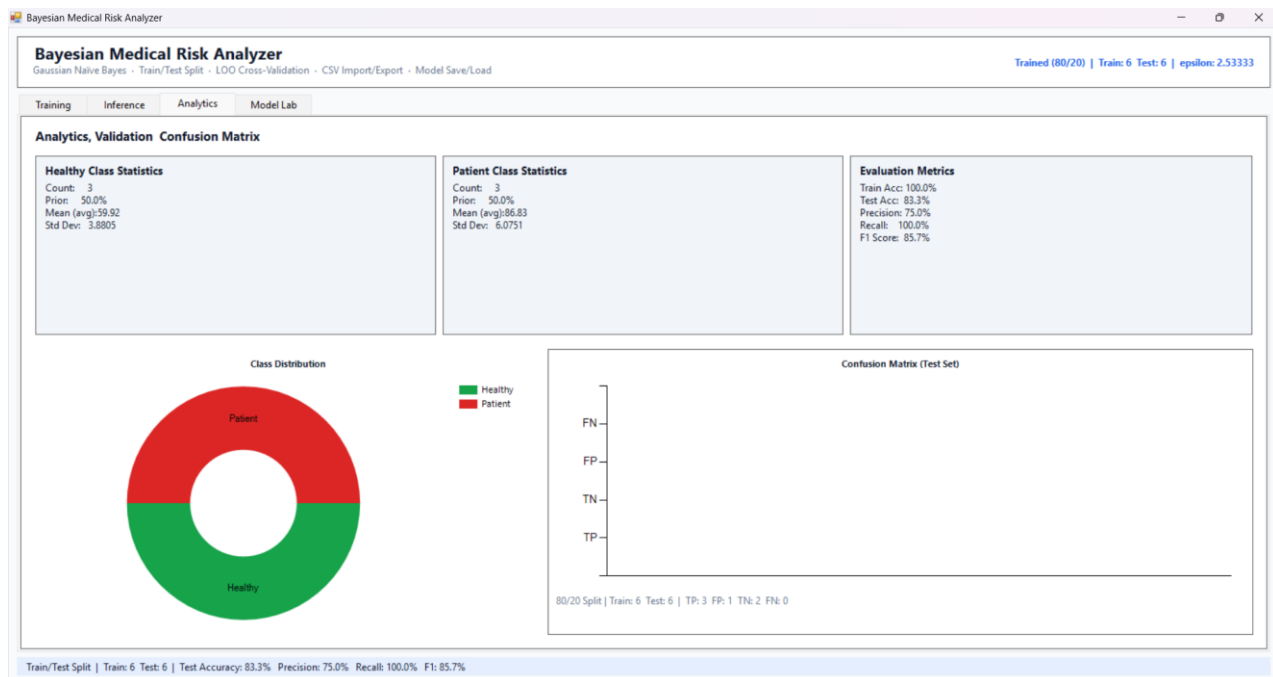


Figure 3. Analytics Tab showing Healthy and Patient class statistics, evaluation metrics, the class-distribution doughnut chart, and the confusion matrix bar chart.

The Analytics tab provides a comprehensive post-training validation dashboard organized in two rows:

Top row — Three statistic cards:

- Healthy Class Statistics: sample count, prior probability, average mean, and average standard deviation across all four features.
- Patient Class Statistics: identical fields for the at-risk class.
- Evaluation Metrics: Train Accuracy, Test Accuracy, Precision, Recall, and F1 Score (Train/Test Split mode) or LOO Accuracy, Precision, Recall, F1, and skipped-folds count (LOO mode).

Bottom row — Two charts:

- Class Distribution (doughnut chart): proportional representation of Healthy vs. Patient samples in the current dataset.
- Confusion Matrix — Test Set (horizontal bar chart): bars for TP (dark green), TN (light green), FP (red), FN (orange), with the exact split counts and confusion-matrix values shown in a label below.

## 5.4 Model Lab Tab

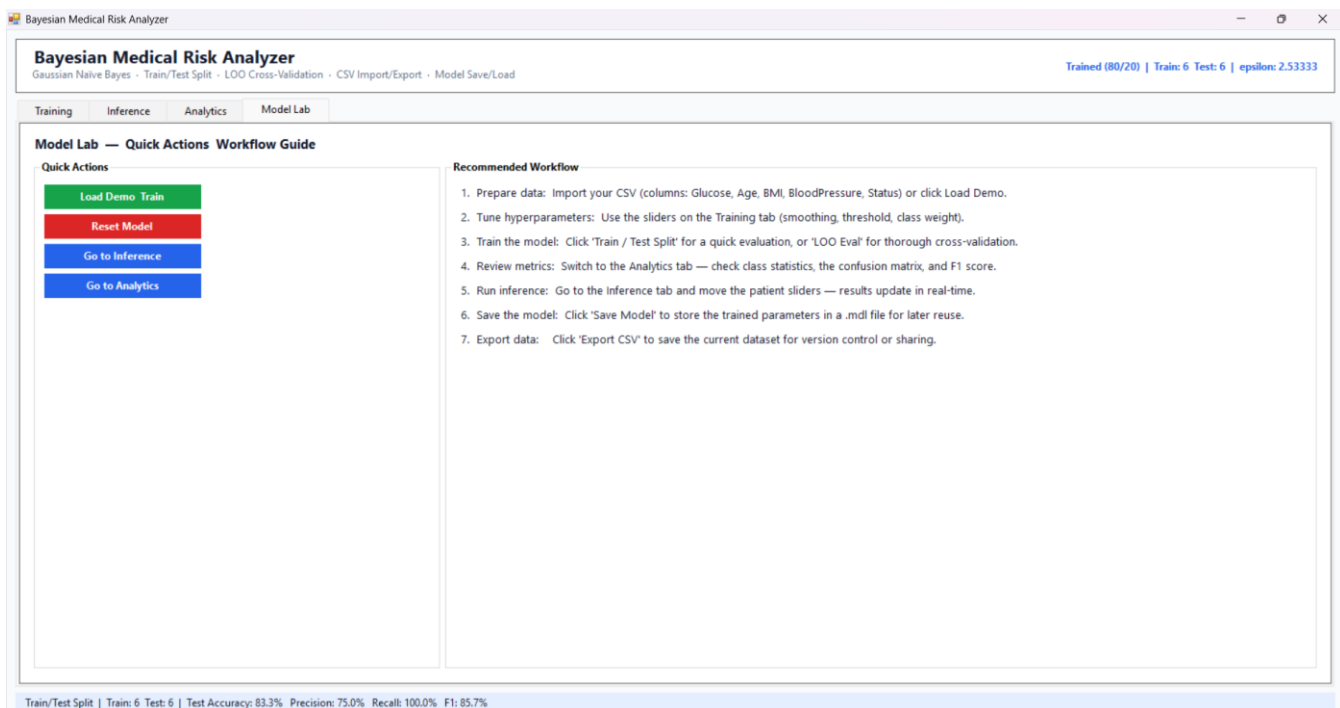


Figure 4. Model Lab Tab showing the Quick Actions panel (left) and the Recommended Workflow numbered guide (right).

The Model Lab tab serves as a quick-access control panel and usage guide. It is divided into two panels:

- Quick Actions (left) — Four shortcut buttons: Load Demo & Train (executes LoadDemoData followed by TrainWithSplit), Reset Model, Go to Inference, and Go to Analytics.
- Recommended Workflow (right) — A seven-step numbered guide that walks the user through the complete application workflow from data preparation to model export.

## 6. Hyperparameter Guide

All hyperparameters are exposed via TrackBar sliders on the Training tab and take effect on the next training invocation. Their internal values are converted from integer slider positions to floating-point hyperparameter values as follows:

| Hyperparameter       | Slider Range | Conversion Formula           | Effect on Model                                                                                                                                                          |
|----------------------|--------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Manual Patient Prior | 1 – 99 (%)   | value / 100.0                | Overrides empirical class frequency. Active only when 'Use empirical prior' is unchecked. Setting > 0.5 biases the model toward predicting Patient.                      |
| Variance Smoothing   | 0 – 100      | value / 1000.0 ( $\lambda$ ) | Added as $\lambda \cdot \text{globalMaxVariance}$ to every feature variance. Higher values reduce overfitting to small datasets at the cost of discriminative sharpness. |
| Decision Threshold   | 10 – 90 (%)  | value / 100.0                | Posterior probability above which a sample is classified as Patient. Lowering the threshold increases recall (sensitivity) at the cost of precision.                     |
| Patient Class Weight | 50 – 200     | value / 100.0 (weight)       | Multiplied into the Patient log-prior before normalization. Values > 1.0 increase sensitivity (favor Patient prediction); values < 1.0 increase specificity.             |

The empirical prior checkbox, when enabled, sets  $P(c) = N^c / N$  for each class, making the Manual Patient Prior slider inactive (displayed as 'Auto').

## 7. Dataset Specification

### 7.1 Feature Columns

| Column         | CSV Header    | Type   | Range (typical)           | Clinical Meaning                     |
|----------------|---------------|--------|---------------------------|--------------------------------------|
| Glucose        | Glucose       | double | 50 – 300 mg/dL            | Fasting plasma glucose concentration |
| Age            | Age           | double | 18 – 90 years             | Patient age                          |
| BMI            | BMI           | double | 15 – 50 kg/m <sup>2</sup> | Body-mass index                      |
| Blood Pressure | BloodPressure | double | 80 – 200 mmHg             | Systolic blood pressure              |
| Label          | Status        | string | Healthy / Patient         | Binary class label                   |

### 7.2 CSV Format

The expected CSV format has a single header row followed by data rows:

```
Glucose,Age,BMI,BloodPressure,Status  
72,24,21.0,110,Healthy  
118,41,28.0,138,Patient
```

The Status column is case-insensitive; both 'healthy'/'patient' and 'Healthy'/'Patient' are accepted. Any other value raises a `FormatException` during import.

### 7.3 Built-in Demo Dataset

Clicking Load Demo populates the grid with 12 synthetically generated but medically plausible samples — 6 Healthy and 6 Patient — with clearly separated feature distributions:

| Class   | Glucose (mg/dL)              | Age (yr)               | BMI   | BP (mmHg) |
|---------|------------------------------|------------------------|-------|-----------|
| Healthy | 72, 78, 84, 90, 98, 106      | 24, 26, 29, 31, 33, 36 | 21–26 | 110–130   |
| Patient | 118, 126, 138, 150, 164, 180 | 41, 44, 47, 50, 54, 58 | 28–41 | 138–178   |

This dataset is designed to produce clean, interpretable results suitable for classroom demonstration. All features follow monotonically increasing progressions per class.

## 7.4 Validation Constraints

The application enforces the following constraints before initiating any training:

- Minimum 6 total samples (AppConstants.MinTotalSamples).
- Minimum 3 samples per class (AppConstants.MinSamplesPerClass), in both the full dataset and each split partition.
- Both class labels (Healthy, Patient) must be present in the training set.
- All four feature cells must contain parseable numeric values (double, invariant or current culture).

## 8. Model Persistence (. mdl File Format)

Trained models are serialized to a plain-text file with the extension. mdl using CultureInfo.InvariantCulture for all floating-point values (ensuring locale-independent portability). The file begins with a version header and contains global parameters followed by one block per class.

### 8.1 File Structure

```
BayesianAnalyzerModelV2    ← version tag
<epsilon>                  ← smoothing constant (float)
<featureCount>             ← number of features (int)
<useEmpiricalPrior>        ← 1 or 0
<manualPositivePrior>      ← float
<varianceSmoothing>        ← float ( $\lambda$ )
<decisionThreshold>        ← float
<positiveClassWeight>      ← float
<positiveLabel>            ← 'Patient'
<negativeLabel>            ← 'Healthy'
CLASS|<label>|<count>|<prior> ← one block per class
MEAN|v1;v2;v3;v4
VAR|v1;v2;v3;v4
STD|v1;v2;v3;v4
```

### 8.2 Loading

During load, the classifier is reconstructed from the stored parameters and injected directly into DiagnosticSession via reflection on the private \_classifier field. This avoids exposing a public setter while still allowing deserialization. After loading, all UI panels and charts are refreshed automatically.

## 9. Validation Results (Demo Dataset)

The following results were obtained by loading the built-in 12-sample demo dataset and executing Train / Test Split (80/20 stratified) with default hyperparameters ( $\epsilon$ -smoothing = 0.0250, threshold = 50%, class weight = 1.00 $\times$ , empirical prior enabled).

### 9.1 Train/Test Split Results

| Metric            | Training Set (n = 6) | Test Set (n = 6)        |
|-------------------|----------------------|-------------------------|
| Accuracy          | 100.0 %              | 83.3 %                  |
| Precision         | 100.0 %              | 75.0 %                  |
| Recall            | 100.0 %              | 100.0 %                 |
| F1 Score          | 100.0 %              | 85.7 %                  |
| TP / TN / FP / FN | —                    | TP: 3 FP: 1 TN: 2 FN: 0 |

### 9.2 Class Statistics After Training

| Statistic     | Healthy Class | Patient Class |
|---------------|---------------|---------------|
| Sample Count  | 3             | 3             |
| Prior         | 50.0 %        | 50.0 %        |
| Mean (avg)    | 59.92         | 86.83         |
| Std Dev (avg) | 3.8805        | 6.0751        |
| Epsilon       | 2.53333       | 2.53333       |

### 9.3 Inference Example

With the inference sliders set to Glucose = 120 mg/dL, Age = 25 yr, BMI = 25, Blood Pressure = 120 mmHg, the classifier outputs:

- Predicted class: PATIENT
- Patient posterior: 71.91 %
- Healthy posterior: 28.09 %
- Outlier status: Not an outlier (z-score within bounds)

The P(Patient) vs Glucose chart shows a sigmoid-shaped decision boundary, transitioning from near-zero to near-one around 110–130 mg/dL, consistent with the class separation in the demo dataset.

### 9.4 Result Interpretation

The high recall (100 % on test set) indicates the model captures all true positive (Patient) cases, with one false positive (a Healthy sample misclassified as Patient). This reflects the typical precision–recall trade-off when the class distributions partially overlap in the



feature space. In a medical screening context, high recall is preferred over high precision to minimize missed diagnoses.

## 10. Limitations and Future Work

### 10.1 Known Limitations

- Feature independence assumption — The Naïve Bayes model assumes all features are conditionally independent given the class. Clinical features such as BMI and blood pressure are moderately correlated, which may slightly degrade calibration on real-world data.
- Fixed feature set — The current implementation is hard-coded to exactly four features (Glucose, Age, BMI, BloodPressure). Supporting variable-length feature vectors would require a schema-aware CSV parser and dynamic slider generation.
- Binary classification only — The model supports exactly two class labels ('Healthy' / 'Patient'). Multi-class extension would require changing the argmax decision rule and the confusion matrix visualization.
- Small demo dataset — The built-in demo dataset has 12 samples, sufficient for demonstrating functionality but insufficient for drawing clinically valid conclusions. Real-world deployment would require a dataset of at least several hundred samples.
- No k-fold cross-validation — Only LOO and a single 80/20 split are implemented. Stratified k-fold would provide more robust generalization estimates.
- No feature normalization — The classifier operates on raw feature values. Although Gaussian NB is invariant to scaling (since it estimates per-feature distributions), normalization would improve the consistency of the P(Patient) vs Glucose visualization.
- Reflection-based model loading — Injecting the deserialized classifier via reflection on `DiagnosticSession._classifier` is fragile; renaming the private field would silently break model loading.

### 10.2 Suggested Extensions

- Replace the four hard-coded features with a dynamic schema read from the CSV header row.
- Add stratified k-fold cross-validation (e.g.,  $k = 5$  or  $10$ ) with aggregated metrics.
- Implement a ROC curve (TPR vs FPR at varying thresholds) on the Analytics tab.
- Integrate feature correlation heatmap to visualize violations of the independence assumption.

- Add SHAP-based or likelihood-ratio feature importance scores to explain individual predictions.
- Extend to multi-class classification using one-vs-rest or softmax normalization.
- Export trained model as ONNX or PMML for interoperability with other ML tools.

## 11. References

---

- [1] Mitchell, T. M. (1997). Machine Learning. McGraw-Hill. Chapter 6: Bayesian Learning.
- [2] Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer. §4.2: Probabilistic Generative Models.
- [3] Murphy, K. P. (2012). Machine Learning: A Probabilistic Perspective. MIT Press. Chapter 3: Generative Models for Discrete Data.
- [4] Zhang, H. (2004). The Optimality of Naive Bayes. FLAIRS Conference Proceedings.
- [5] Powers, D. M. W. (2011). Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness & Correlation. Journal of Machine Learning Technologies, 2(1), 37–63.
- [6] Microsoft Corporation. (2024). Windows Forms documentation (.NET). <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/>
- [7] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. (Reference implementation of Gaussian NB for comparison.)